

SUPPLYGUARD AI

PROJECT DOCUMENTATION

AI-Powered Supply Chain Disruption Assistant
& Fleet Utilisation Optimizer

1. PROBLEM STATEMENT

Modern logistics companies operate large fleets, multiple warehouses, thousands of shipments and many delivery routes. Unexpected events such as highway closures, vehicle breakdowns, heavy rainfall, warehouse delays and demand spikes can cause delivery delays and additional operating costs.

Traditional fleet systems may show that a route is blocked, but managers also need to know which shipments and vehicles are affected, how much delay is expected, which shipments are urgent, which vehicle should be reassigned, which route should be selected, and how costs can be reduced.

2. PROPOSED SOLUTION

SupplyGuard AI is an AI-assisted logistics decision-support platform.

Data → Detect Disruption → Analyse Impact → Predict Delay
→ Optimise Fleet/Routes → Recommend Action

The system does not only report a disruption. It recommends what the logistics manager should do next.

3. KEY FEATURES

- Fleet dashboard
- Disruption simulator
- Shipment impact analysis
- Delay prediction
- Fleet utilisation optimiser
- Route optimisation
- AI recommendation engine
- What-If simulator
- Natural-language logistics assistant

SUPPLYGUARD AI

PROJECT DOCUMENTATION — AI & OPTIMISATION

4. EXAMPLE NH-48 SCENARIO

A simulated NH-48 disruption affects routes between Ahmedabad, Vadodara and Surat.

Example impact:

- 12 shipments affected
- 3 vehicles affected
- Estimated additional delay: 150 minutes
- Estimated additional cost: ₹8,400

AI RECOMMENDED ACTION

- 1. Reassign V03 to critical shipment S17.**
- 2. Use alternate Route B.**
- 3. Prioritise S17 because its deadline is approaching.**

BEFORE → AFTER

Average delay 150 min → 35 min

At-risk shipments 12 → 3

Additional cost ₹8,400 → ₹3,600

Fleet utilisation 68% → 86%

5. AI / ML APPROACH

Delay Prediction:

Distance + traffic + weather + vehicle speed + route type

→ predicted travel time and delay.

Vehicle Assignment considers:

- Distance from shipment
- Remaining capacity
- Shipment priority
- Route compatibility
- Vehicle availability
- Utilisation

Priority:

Critical / Medical → Highest

Urgent → High

High → Medium-high

Normal → Standard

SUPPLYGUARD AI

PROJECT DOCUMENTATION — ARCHITECTURE & DEMO

6. SYSTEM ARCHITECTURE

USER / LOGISTICS MANAGER

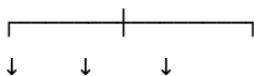


WEB DASHBOARD

HTML + CSS + JavaScript



FLASK API



Disruption Predictor

Optimizer

Engine



Recommendation Engine



AI ACTION / WHAT-IF RESULT

7. TECHNOLOGY STACK

Frontend:

HTML5, CSS3, JavaScript, Chart.js, Leaflet.js

Backend:

Python, Flask

Data & Analytics:

Pandas, CSV / SQLite, Scikit-learn

Testing:

Python unit tests + GitHub Actions

8. WHAT-IF SIMULATOR

Example: Vehicle V05 breaks down.

Before:

10 vehicles | 25 shipments | 28 min average delay

After simulation:

9 available vehicles | 25 shipments | 41 min average delay

Example AI recommendation:

“Reassign V08 to S12 and V03 to S17. Prioritise urgent shipments and use an available alternate route.”

9. EXPECTED BENEFITS

SUPPLYGUARD AI

PROJECT DOCUMENTATION — HACKATHON SUBMISSION

10. USER WORKFLOW

1. Open the SupplyGuard AI dashboard.
2. Review fleet and shipment status.
3. Detect or simulate a disruption.
4. Identify affected routes, vehicles and shipments.
5. Estimate delay and cost impact.
6. Run the optimiser.
7. Review the recommended action.
8. Compare before/after results.
9. Run a What-If scenario.
10. Ask the AI assistant for an explanation.

11. HACKATHON ALIGNMENT

Disruption detection	→ Disruption Simulator
Affected shipments	→ Impact Analysis
Delay prediction	→ Delay Prediction
Vehicle reassignment	→ Fleet Optimizer
Alternative routes	→ Route Optimisation
Cost reduction	→ Before/After Analysis
Fleet utilisation	→ Utilisation Score
Urgent shipments	→ Priority Engine
Decision support	→ AI Recommendation
Scenario planning	→ What-If Simulator

12. HOW TO RUN

Frontend:

```
cd frontend
```

```
python -m http.server 5500
```

Then open:

<http://localhost:5500>

Backend:

```
pip install -r requirements.txt
```

```
python backend/app.py
```

13. FUTURE SCOPE

- Real-time GPS tracking
- Live traffic and weather APIs
- IoT vehicle sensors
- Warehouse integrations